# TaskFlow — AI Project & Task Management

Complete Technical Architecture, Stack Breakdown & User Guide

## 1. Executive Summary & Purpose

**TaskFlow** is an enterprise-grade, information-dense SaaS software platform engineered for modern software teams. It provides full-lifecycle project planning, task management, team workload tracking, and automated Artificial Intelligence features that solve everyday bottlenecks.

**Design Philosophy:** Inspired by tools like *Linear* and *Jira*, TaskFlow avoids flashy decorative effects, cluttered animations, and generic AI templates. Instead, it prioritizes a clean 8-point grid, soft neutral surfaces, clear typography, high information density, and strict role-based data isolation.

## 2. Complete Technology Stack & Rationale

Every technology in the TaskFlow stack was chosen intentionally for performance, maintainability, type safety, and production readiness.

| Layer | Technology | Version | Simple English Explanation & Why Chosen |
|---|---|---|---|
| **Frontend UI** | React + TypeScript | React 18<br>TS 5.6 | **React** renders dynamic, interactive web interfaces. **TypeScript** adds static types to prevent bugs before code ever runs in the browser. |
| **Build Tool** | Vite | 5.4 | Next-generation build tool with instant Hot Module Replacement (HMR) and optimized rollup production bundles. |
| **Styling** | Tailwind CSS | 3.4 | Utility-first styling for a clean design token system (colors, borders, typography, 8-point spacing) without CSS bloat. |
| **Visuals** | Lucide React + Recharts | Latest | Accessible, modern SVG icons and purposeful data visualizations (burndown, completion rates, workload distribution). |
| **Backend Core** | Java + Spring Boot | Java 21<br>Boot 3.3.4 | **Java 21 LTS** provides top performance and stability. **Spring Boot** provides an enterprise modular architecture with auto-configured web servers. |
| **Database ORM** | Spring Data JPA / Hibernate | 6.5 | Maps Java domain models to database tables and handles database queries safely without manual error-prone SQL. |
| **Security** | Spring Security + JWT | JJWT 0.12 | Stateless JSON Web Tokens with HMAC cryptography. Role-based access control protecting all REST endpoints. |
| **Password Hash** | BCrypt | Strength 12 | Adaptive cryptographic one-way hashing with automatic salting to guard user credentials against database leaks. |
| **Database** | MySQL 8.0 / H2 Memory | MySQL 8<br>H2 Mem | **MySQL** for persistent ACID production storage. **H2** in-memory database profile for zero-config instant local execution. |
| **AI Engine** | Spring AI + Google Gemini | 1.0.0-M3<br>Gemini 1.5 | **Spring AI** integrates LLMs with automated fallback heuristics. **Google Gemini** handles live task generation and assistant chat. |

| | | | |
|---|---|---|---|
| **API Docs** | SpringDoc OpenAPI | 2.6.0 | Auto-generates interactive Swagger UI documentation at /swagger-ui.html for testing every REST endpoint. |

## 3. Modular Backend Architecture

The backend follows a **clean modular monolith pattern**. Each functional area is isolated into its own package with strict separation between presentation, business logic, and data access layers:

- **Controller Layer (@RestController):** Handles incoming HTTP requests, validates request payloads with Jakarta validation, enforces role permissions, and returns standardized ApiResponse<T>.
- **Service Layer (@Service):** Houses all business rules, authorization assertions, circular dependency checks, and audit logging within @Transactional boundaries.
- **Repository Layer (@Repository):** Extends Spring Data JPA interfaces, providing indexed queries, pagination, and sorting.
- **Entity Layer (@Entity):** Defines relational schemas, constraints, foreign keys, and automatic timestamp auditing (createdAt, updatedAt).
- **DTO Layer:** Isolates database entities from external network contracts to prevent data leakage and decouple schema evolution from the frontend.
- **Global Exception Handler:** Intercepts all validation, security, and runtime errors, returning user-friendly error maps with accurate HTTP status codes (400, 401, 403, 404, 500).

## 4. Directed Acyclic Graph (DAG) Task Dependencies

In complex engineering projects, tasks frequently depend on prerequisite tasks (e.g., UI implementation cannot begin before API endpoints are finished). TaskFlow models task dependencies as a **Directed Acyclic Graph (DAG)**. Before any new dependency link (A → B) is saved:

- The system executes a **cycle detection algorithm (Breadth-First Search / Depth-First Search)** to verify whether B can already reach A through existing links.
- If adding the link would create a circular loop (A → B → C → A), the transaction is immediately rejected with a 400 Bad Request and a clear explanatory message.

## 5. The Five Real-World AI Capabilities

Rather than a gimmicky chatbot, TaskFlow incorporates Artificial Intelligence directly into core project management workflows:

**1. AI Task Plan Generator:** The project manager inputs project objectives, technology stack, target deadline, and team context. The AI generates a structured, estimated task breakdown including title, technical description, estimated duration, suggested developer role, prerequisite dependencies, and Given/When/Then acceptance criteria.

**2. Human-in-the-Loop Review & Approval:** AI suggestions are NEVER saved directly to the database. Instead, they are presented in a reviewable, editable table where the manager can modify estimates, uncheck unwanted tasks, and click 'Approve & Create Tasks' to commit them to the sprint.

**3. Transparent Project Risk Analysis:** Analyzes real project metrics—such as overdue tasks, blocked items, critical tasks near deadlines, and incomplete prerequisites. It calculates an explainable risk score (0–100) and provides actionable recommendations to get the project back on track.

**4. Executive Progress Summary:** Distinguishes verified database facts (e.g., 40% complete, 1 blocked, 2 in progress) from AI recommendations, giving leadership a quick, hallucination-free project report.

**5. Task Implementation Plan Generator:** Developers can click 'Generate Plan' on any task to receive an architectural step-by-step roadmap, including required environment prerequisites, subtask phases, testing strategies, and completion criteria.

> ***Dual-Mode AI Engine:*** *TaskFlow is equipped with a dual-mode engine. When an API key is connected (Google Gemini or OpenAI), it generates rich, generative text. When running offline or without an API key, the proprietary Heuristic Fallback Engine takes over, providing accurate, structured answers from local database records without failing.*

## 6. Relational Database Schema & Entities

The relational database schema is normalized with primary keys, foreign keys, unique constraints, and audit columns:

| Entity | Table Name | Key Columns | Relationships & Constraints |
|---|---|---|---|
| **User** | users | id, email, password_hash, role, name | Unique constraint on email. Roles: MANAGER, EMPLOYEE. |
| **Project** | projects | id, name, description, status, start_date, end_date, manager_id, archived | Many-to-One with users (manager_id). Status: PLANNING, ACTIVE, ON_HOLD, COMPLETED, CANCELLED. |
| **Task** | tasks | id, title, description, status, priority, progress, due_date, project_id, assignee_id | Many-to-One with projects and users. Status: TODO, IN_PROGRESS, IN_REVIEW, COMPLETED, BLOCKED. Priority: LOW, MEDIUM, HIGH, CRITICAL. |
| **Task Dependency** | task_dependencies | id, task_id, depends_on_id, created_at | Self-referencing Many-to-One on tasks. Unique constraint on (task_id, depends_on_id). |
| **Task Comment** | task_comments | id, task_id, author_id, content, created_at | Threaded comments linked to tasks and user authors. |
| **Task Activity** | task_activities | id, task_id, actor_id, action, old_value, new_value, created_at | Immutable audit log capturing status transitions, progress edits, and AI creations. |

## 7. Role-Based Access Control (RBAC) Matrix

Security is enforced at the controller method level via @PreAuthorize annotations:

| Capability / Operation | Project Manager (MANAGER) | Team Member (EMPLOYEE) |
|---|---|---|
| **View Dashboard & Assigned Work** | Full workspace overview | Personal assigned tasks & deadlines |
| **Create / Edit / Archive Projects** | Allowed (Full CRUD) | Read-only access |
| **Create / Delete Tasks** | Allowed for managed projects | Denied (403 Forbidden) |
| **Update Task Status & Progress** | Allowed | Allowed only for assigned tasks |
| **Manage Task Dependencies** | Allowed | Denied |
| **Generate & Approve AI Task Plans** | Allowed | Denied |
| **Generate Task Implementation Plan** | Allowed | Allowed for assigned tasks |
| **Self-Registration Restriction** | System created only | Default self-registration role |

## 8. How to Run, Test, and Verify

TaskFlow can be booted in two simple terminal commands without complex setup:

• **Terminal 1 — Spring Boot Backend:**
cd "d:\Project 1\backend"
.\mvnw.cmd spring-boot:run "-Dspring-boot.run.profiles=h2"
*(Starts Tomcat on port 8080 with in-memory database and auto-seeds all production demo data in 6 seconds).*

• **Terminal 2 — React Vite Frontend:**
cd "d:\Project 1\frontend"
npm run dev
*(Starts Vite dev server on port 5173 with proxy configured to backend port 8080).*

• **Pre-Seeded Demo Accounts:**
• Manager: manager@taskflow.com / Password123!
• Employee: alex@taskflow.com / Password123!

• **Key Endpoints & URLs:**
• Web App: http://localhost:5173
• Interactive Swagger API Docs: http://localhost:8080/swagger-ui.html
• H2 Database Console: http://localhost:8080/h2-console (JDBC URL: jdbc:h2:mem:taskflow_db, User: sa, Pass: empty)